Azure Synapse Analytics ETL Pipeline Project

Name: Vemula Raghunath Goud

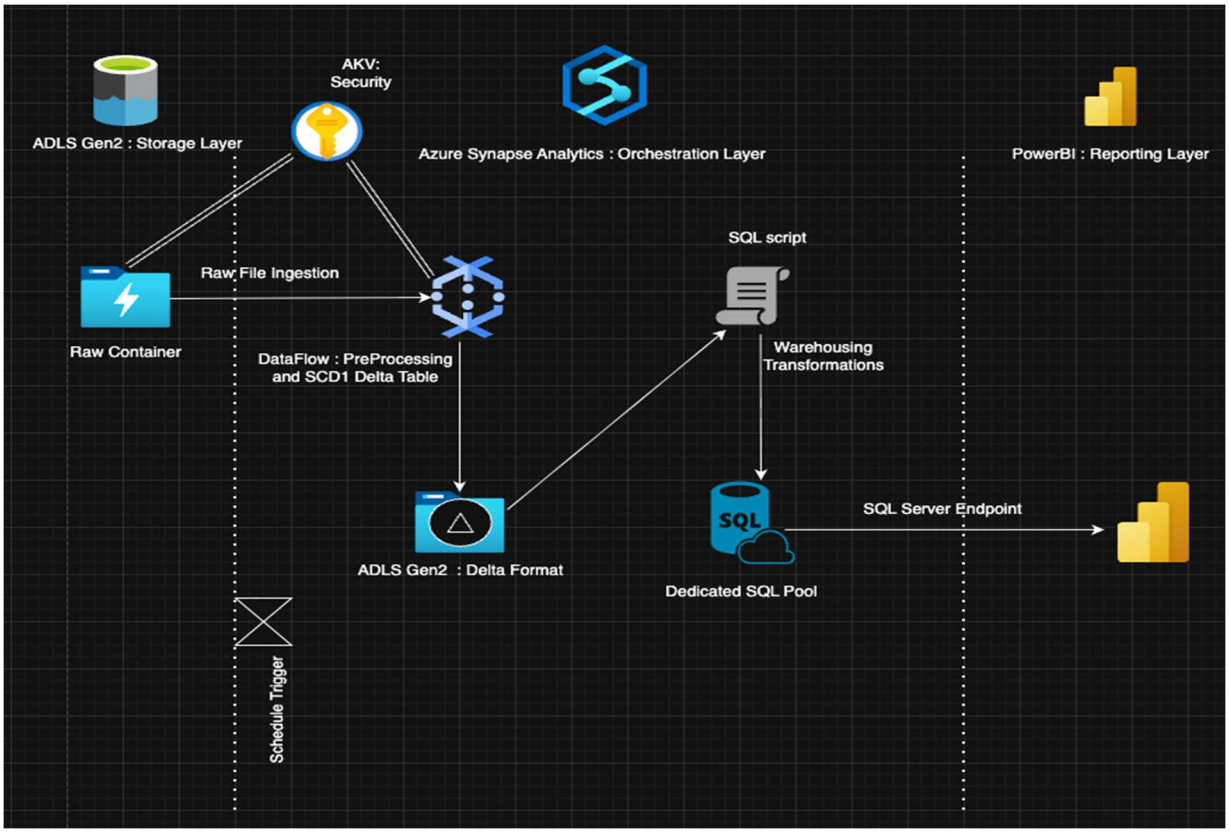
Title: End-to-End Data Pipeline with SCD Type 1 using Azure Synapse Analytics

Contents

1. Project Overview 2
2. Technologies Used 2
3. Data Layers 3
4. SCD Type 1 Implementation 5
5. Serverless SQL Layer 10
6. Data Warehouse Design (Dedicated SQL Pool) 14
7. Data Loading Strategy 18
8. Data Validation 18
9. Challenges Faced & Learnings 19
10. Conclusion 19

Architecture:

Medallion Architecture (Raw → Silver → Warehouse)



1. Project Overview

Objective:
To design and implement a scalable data pipeline that ingests transactional data, applies SCD Type 1 logic using Synapse Data Flow, and loads curated data into a warehouse model in Synapse Dedicated SQL Pool.

- Process sales dataset from raw files
- Remove duplicates, preprocess and clean data
- Implement **SCD Type 1** to maintain latest state
- Build a **dimensional model (Fact + Dimensions)**
- Enable analytical reporting using SQL

Kaggle CSV
↓
ADLS Gen2 (Raw Layer)
↓
Synapse Data Flow (SCD1 Logic)
↓
ADLS Gen2 (Silver Delta)
↓
Serverless SQL (Views)
↓
CETAS (Export to Parquet)
↓
Dedicated SQL Pool (Warehouse Tables)

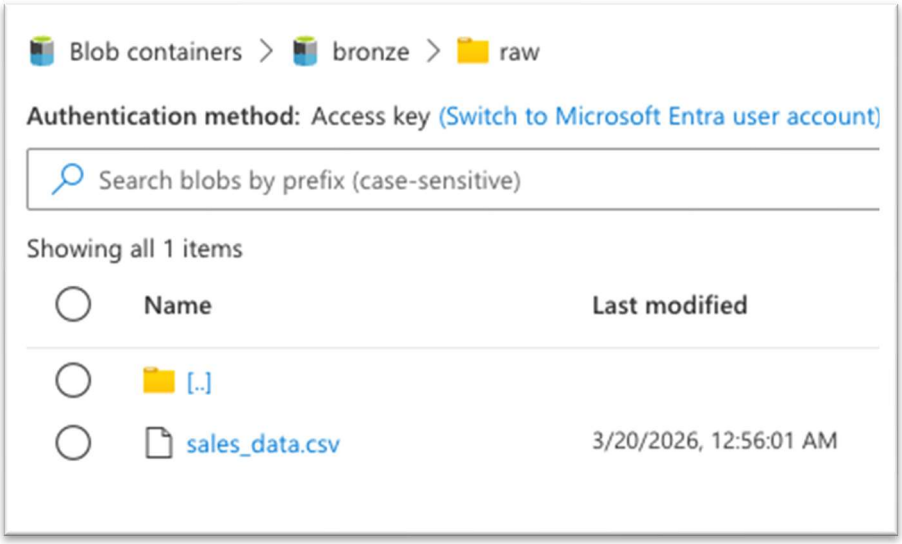
2. Technologies Used

Layer	Technology
Ingestion	Manual Upload / ADLS
Storage	ADLS Gen2
Processing	Synapse Data Flow
Format	Delta Lake
Query Engine	Serverless SQL Pool
Warehouse	Dedicated SQL Pool
Language	SQL

3. Data Layers

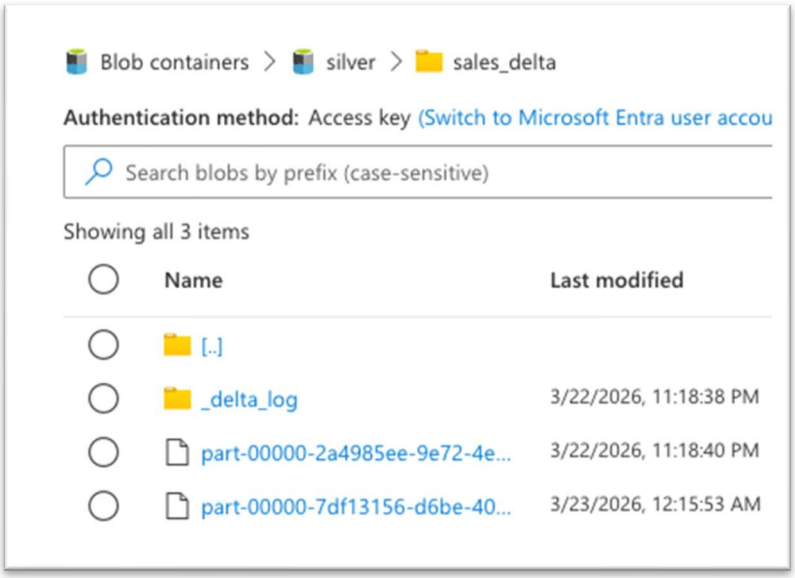
Raw Layer

- Stored original CSV files
- Path: raw/sales_dataset/



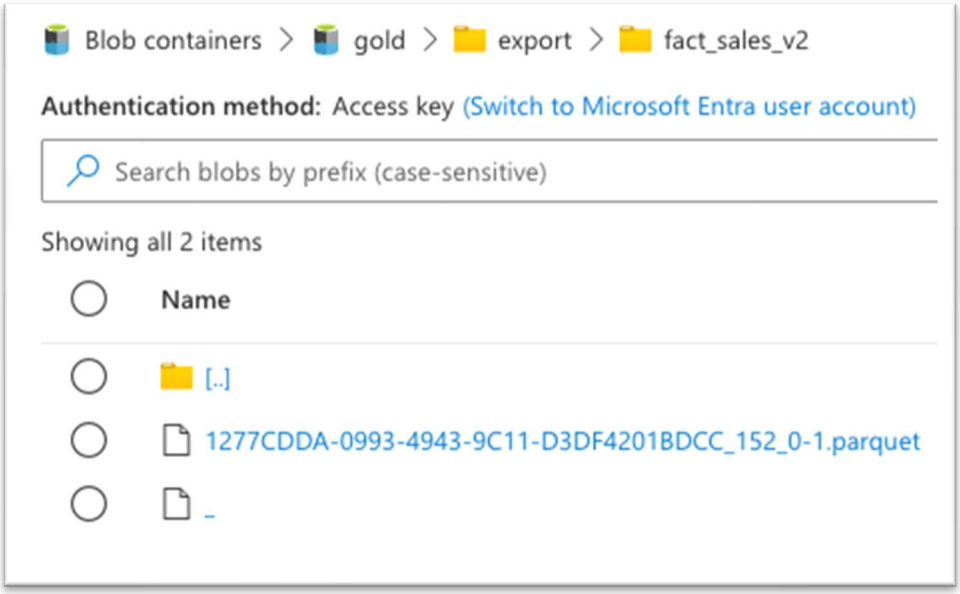
Silver Layer (Delta)

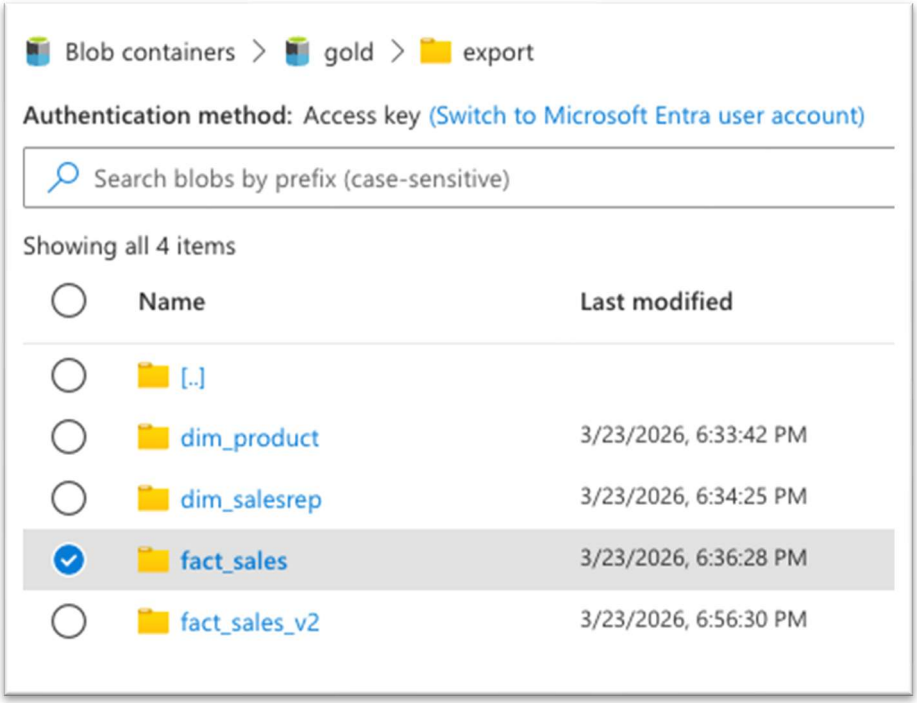
- Cleaned and deduplicated data
- Stored as Delta table
- Path:
  silver/sales_deltaTable/
- Features:
  - Schema enforcement
  - ACID transactions
  - Latest snapshot maintained



Export Layer (Parquet)

- Data exported using CETAS
- Used for loading into Dedicated SQL Pool
- Paths:
    - gold/export/dim_product/
    - gold/export/dim_salesrep/
    - gold/export/fact_sales/



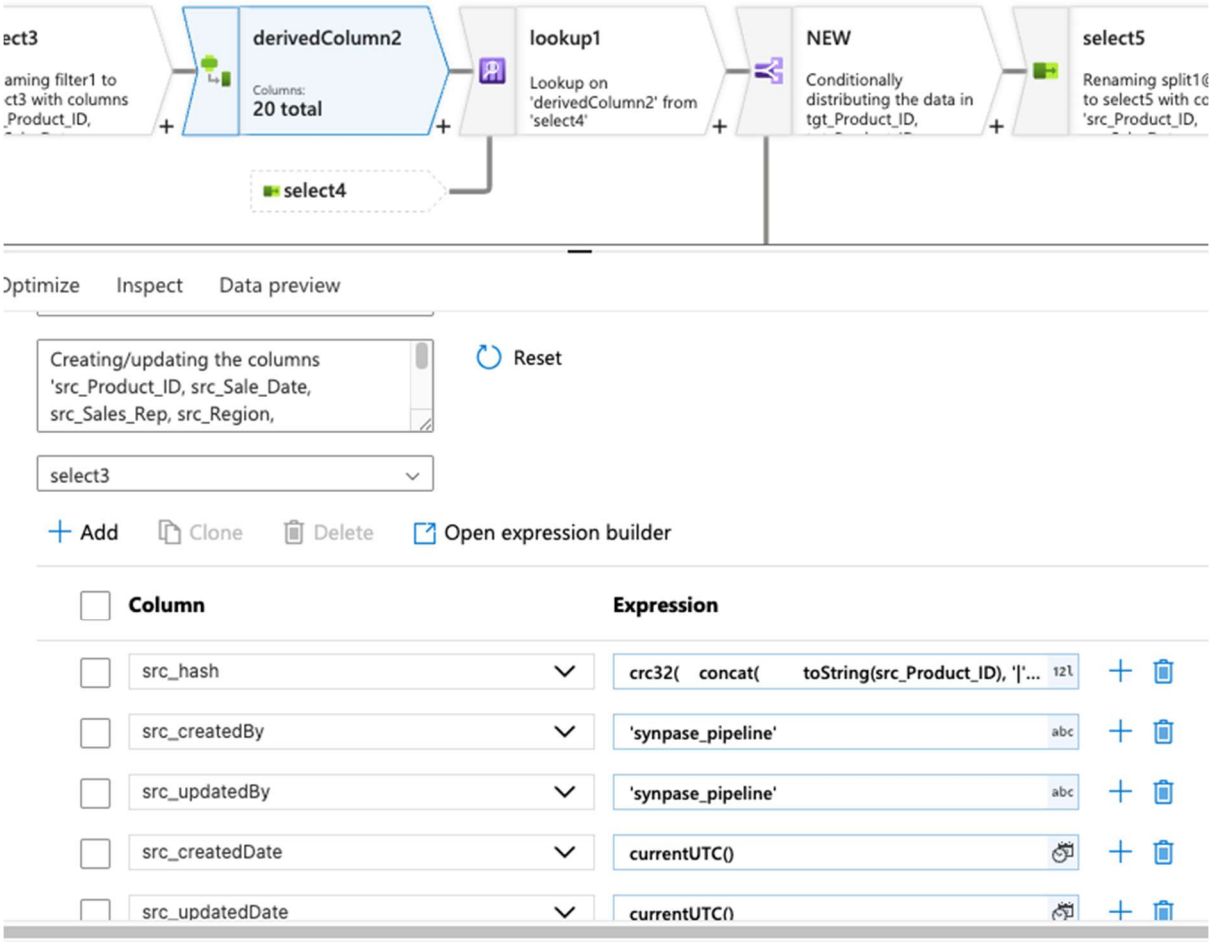


4. SCD Type 1 Implementation

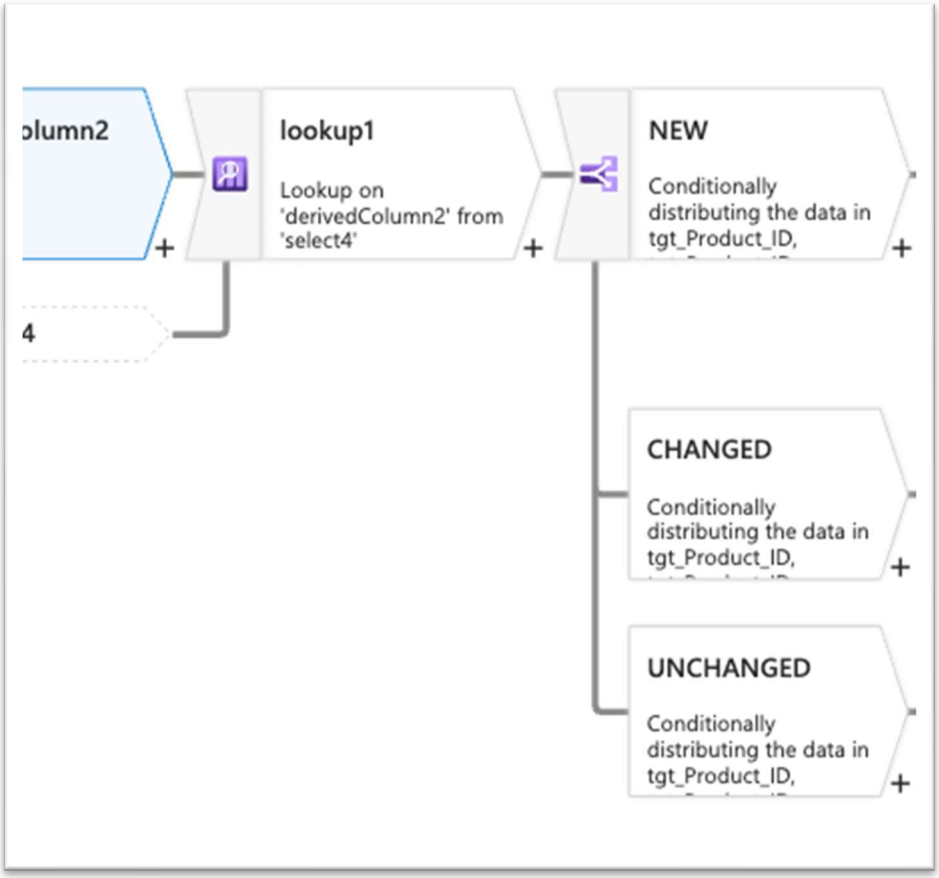
Approach Used:

- Lookup between source and target

- Hash-based change detection

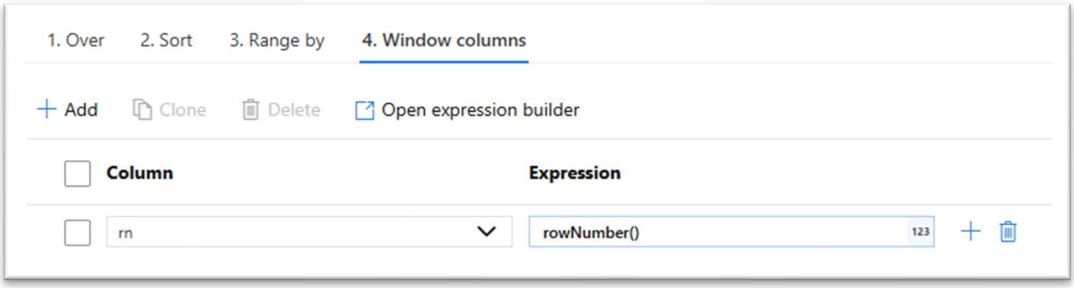


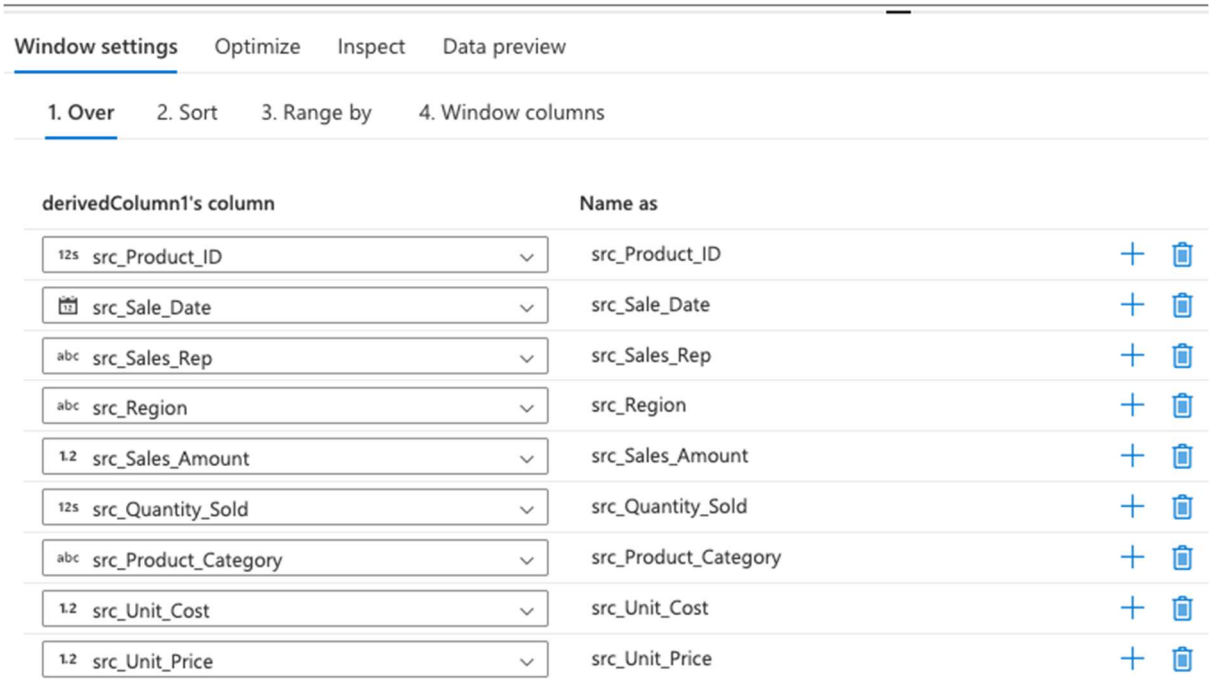
- Conditional split logic



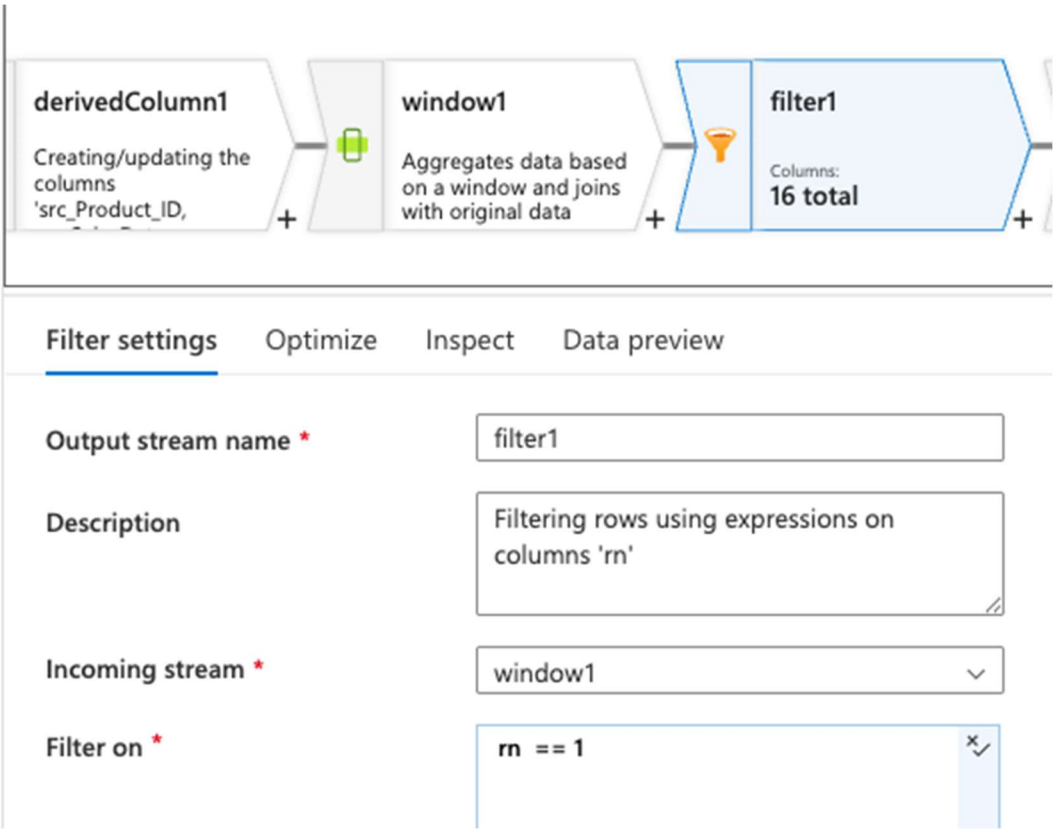
Data Flow Transformations:

1. **Deduplication**
    - Used row_number() window function





Filtered rn = 1



2. **Hash Column**

Column name *

src_hash

Expression

```
crc32(
    concat(
        toString(src_Product_ID), '|',
        coalesce(src_Product_Category, 'NA'), '|',
        toString(src_Unit_Cost), '|',
        toString(src_Unit_Price),'|',
        coalesce(src_Region_and_Sales_Rep)
    )
)
```

3. **Conditional Split**

- New Rows → isNull(tgt_key)
- Changed Rows → hash mismatch
- Unchanged Rows → hash match

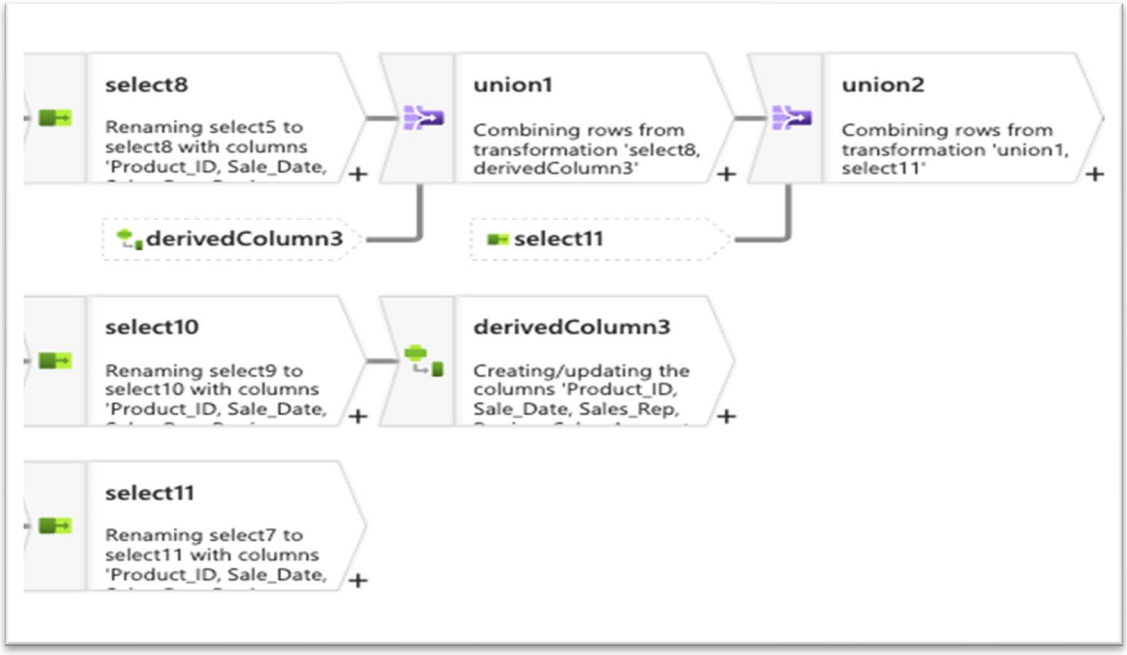
Stream names	Condition
NEW	isNull(tgt_Product_ID)
CHANGED	!isNull(tgt_Product_ID) && src_hash != tgt_hash
UNCHANGED	!isNull(tgt_Product_ID) && src_hash == tgt_hash

4. **Union**

Union by * ⓘ (•) Name ( ) Position

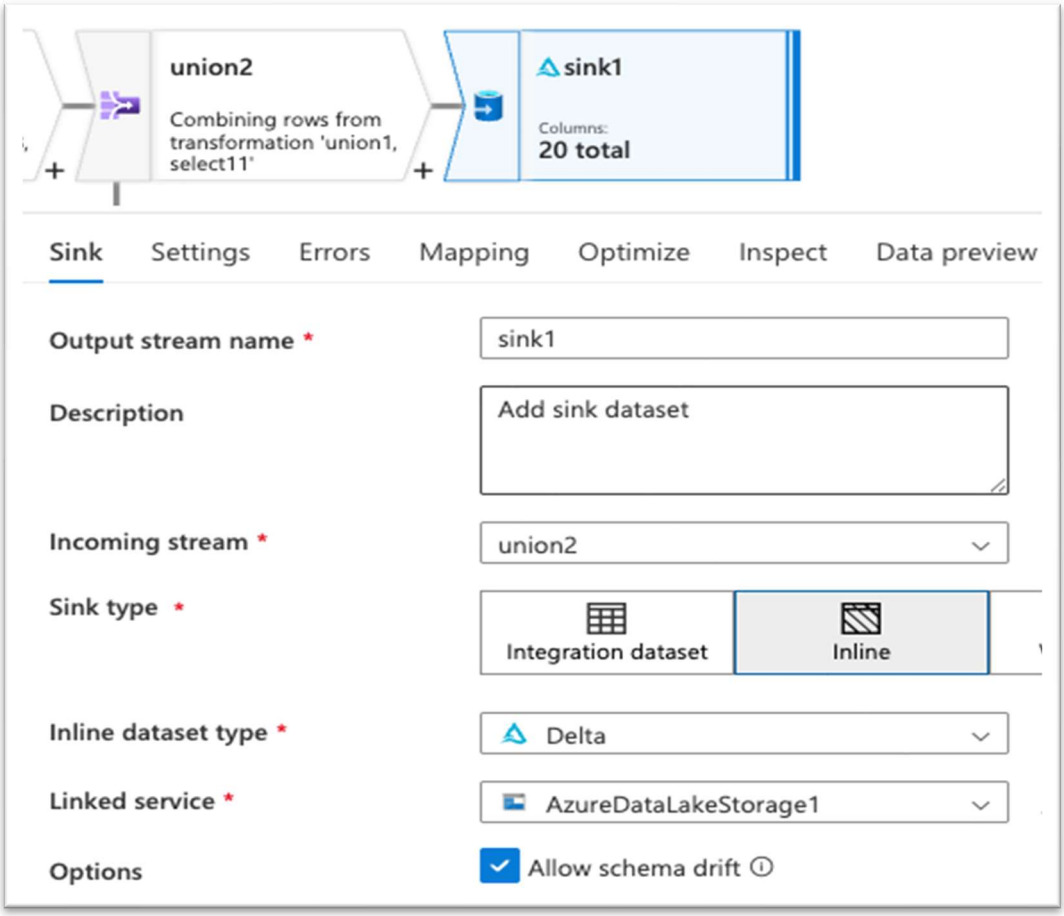
Union with * derivedColumn3

- Combined all streams into final dataset



5. **Sink**

- Written to Delta (overwrite mode)



5. Serverless SQL Layer

Delta Query

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'StrongPassword!123456789';

CREATE DATABASE SCOPED CREDENTIAL SynapseMI
WITH IDENTITY = 'Managed Identity';


CREATE EXTERNAL DATA SOURCE SilverLake_delta11
WITH (
    LOCATION = 'https://adlsg2rag.dfs.core.windows.net/silver/',
    CREDENTIAL = SynapseMI
);


SELECT TOP 10 *
FROM OPENROWSET(
    BULK 'sales_delta/',
    DATA_SOURCE = 'SilverLake_delta11',
    FORMAT = 'DELTA'
) AS rows;
```

Views Created:

```
SELECT TOP 10 *
FROM OPENROWSET(
  BULK 'sales_delta/',
  DATA_SOURCE = 'SilverLake_delta11',
  FORMAT = 'DELTA'
) AS rows;
```

- **vw_sales_delta**

```
CREATE OR ALTER VIEW dbo.vw_sales_delta
AS
SELECT *
FROM OPENROWSET(
  BULK 'sales_delta/',
  DATA_SOURCE = 'SilverLake_delta11',
  FORMAT = 'DELTA'
) AS rows;
GO
```

- **vw_dim_product_src**

```
CREATE OR ALTER VIEW dbo.vw_dim_product_src
AS
SELECT DISTINCT
  Product_ID,
  Product_Category,
  Unit_Cost,
  Unit_Price
FROM dbo.vw_sales_delta
WHERE Product_ID IS NOT NULL;
GO
```

- **vw_dim_salesrep_src**

```
CREATE OR ALTER VIEW dbo.vw_dim_salesrep_src
AS
SELECT
  ROW_NUMBER() OVER (ORDER BY Sales_Rep, Region) AS SalesRep_ID,
  Sales_Rep,
  Region
FROM
(
  SELECT DISTINCT
    Sales_Rep,
    Region
  FROM dbo.vw_sales_delta
  WHERE Sales_Rep IS NOT NULL
) s;
GO
```

- **vw_fact_sales_src**

```
CREATE OR ALTER VIEW dbo.vw_fact_sales_src
AS
SELECT
  CAST(ROW_NUMBER() OVER (
    ORDER BY s.Product_ID, s.Sale_Date, s.Sales_Rep, s.Region
  ) AS BIGINT) AS Sale_ID,
  CAST(s.Product_ID AS INT)       AS Product_ID,
  CAST(r.SalesRep_ID AS INT)      AS SalesRep_ID,
```

```
  s.Sale_Date,
  CAST(s.Sales_Amount AS FLOAT)     AS Sales_Amount,
  CAST(s.Quantity_Sold AS INT)      AS Quantity_Sold,
  s.Customer_Type,
  CAST(s.Discount AS FLOAT)         AS Discount,
  s.Payment_Method,
  s.Sales_Channel,
  s.Region_and_Sales_Rep,
  s.createdBy,
  s.updatedBy,
  s.createdDate,
  s.updatedDate
FROM dbo.vw_sales_delta s
INNER JOIN dbo.vw_dim_salesrep_src r
  ON s.Sales_Rep = r.Sales_Rep
 AND s.Region = r.Region;
GO

SELECT name
FROM sys.databases;

SELECT
  s.name AS schema_name,
  v.name AS view_name
FROM sys.views v
JOIN sys.schemas s
  ON v.schema_id = s.schema_id
ORDER BY s.name, v.name;
```

Creating external file format to export parquet files to ADLS Layer

```
CREATE EXTERNAL FILE FORMAT ParquetFormat
WITH (
  FORMAT_TYPE = PARQUET
);
GO
```

Creating external data source location to define path for ADLS layer.

```
CREATE EXTERNAL DATA SOURCE GoldLake
WITH (
```

```
  LOCATION = 'https://adlsg2rag.dfs.core.windows.net/gold/',
  CREDENTIAL = SynapseMI
);
```

Creating external tables to export data from views to the tables.

```
CREATE EXTERNAL TABLE dbo.ext_dim_product
WITH
(
  LOCATION = 'export/dim_product/',
  DATA_SOURCE = GoldLake,
  FILE_FORMAT = ParquetFormat
)
AS
SELECT
  Product_ID,
  Product_Category,
  Unit_Cost,
  Unit_Price
FROM dbo.vw_dim_product_src;
GO

CREATE EXTERNAL TABLE dbo.ext_dim_salesrep
WITH
(
  LOCATION = 'export/dim_salesrep/',
  DATA_SOURCE = GoldLake,
  FILE_FORMAT = ParquetFormat
)
AS
SELECT
  SalesRep_ID,
  Sales_Rep,
  Region
FROM dbo.vw_dim_salesrep_src;
GO

CREATE EXTERNAL TABLE dbo.ext_fact_sales_v2
WITH
(
```

```
  LOCATION = 'export/fact_sales_v2/',
  DATA_SOURCE = GoldLake,
  FILE_FORMAT = ParquetFormat
)
AS
SELECT
  Sale_ID,
  Product_ID,
  SalesRep_ID,
  Sale_Date,
  Sales_Amount,
  Quantity_Sold,
  Customer_Type,
  Discount,
  Payment_Method,
  Sales_Channel,
  Region_and_Sales_Rep,
  createdBy,
  updatedBy,
  createdDate,
  updatedDate
FROM dbo.vw_fact_sales_src;
GO
```

6. Data Warehouse Design (Dedicated SQL Pool)

Tables Created

1) dim_product

- Distribution: **REPLICATE**

Replicated dimension table

```
IF OBJECT_ID('dbo.dim_product', 'U') IS NOT NULL
  DROP TABLE dbo.dim_product;
GO

CREATE TABLE dbo.dim_product
(
  Product_ID       INT          NOT NULL,
  Product_Category  VARCHAR(100)  NULL,
```

```
  Unit_Cost        FLOAT        NULL,
  Unit_Price       FLOAT        NULL,

  CONSTRAINT PK_dim_product
  PRIMARY KEY NONCLUSTERED (Product_ID) NOT ENFORCED
)
WITH
(
  DISTRIBUTION = REPLICATE,
  CLUSTERED COLUMNSTORE INDEX
);
GO


COPY INTO dbo.dim_product
FROM 'https://adlsg2rag.dfs.core.windows.net/gold/export/dim_product/'
WITH
(
  FILE_TYPE = 'PARQUET',
  CREDENTIAL = (
     IDENTITY = 'Managed Identity'
  )
);
GO
```

2) dim_salesrep

- Distribution: **ROUND_ROBIN**

Round-robin dimension table

```
IF OBJECT_ID('dbo.dim_salesrep', 'U') IS NOT NULL
  DROP TABLE dbo.dim_salesrep;
GO


CREATE TABLE dbo.dim_salesrep
(
  SalesRep_ID      BIGINT           NOT NULL,
  Sales_Rep        VARCHAR(100)   NOT NULL,
  Region           VARCHAR(100)   NULL,

  CONSTRAINT PK_dim_salesrep
```

```
  PRIMARY KEY NONCLUSTERED (SalesRep_ID) NOT ENFORCED
)
WITH
(
  DISTRIBUTION = ROUND_ROBIN,
  CLUSTERED COLUMNSTORE INDEX
);
GO


COPY INTO dbo.dim_salesrep
FROM 'https://adlsg2rag.dfs.core.windows.net/gold/export/dim_salesrep/'
WITH
(
  FILE_TYPE = 'PARQUET',
  CREDENTIAL = (
    IDENTITY = 'Managed Identity'
  )
);
GO
```

3) fact_sales

- Distribution: **HASH(Product_ID)**

Hash-distributed fact table

```
IF OBJECT_ID('dbo.fact_sales', 'U') IS NOT NULL
  DROP TABLE dbo.fact_sales;
GO


CREATE TABLE dbo.fact_sales
(
  Sale_ID          BIGINT       NOT NULL,
  Product_ID       INT          NOT NULL,
  SalesRep_ID      INT          NOT NULL,
  Sale_Date        DATE         NULL,
  Sales_Amount     FLOAT        NULL,
  Quantity_Sold    INT          NULL,
  Customer_Type    VARCHAR(50)  NULL,
  Discount         FLOAT        NULL,
  Payment_Method   VARCHAR(100) NULL,
```

```
  Sales_Channel        VARCHAR(50)    NULL,
  Region_and_Sales_Rep  VARCHAR(200)   NULL,
  createdBy            VARCHAR(100)   NULL,
  updatedBy            VARCHAR(100)   NULL,
  createdDate          DATETIME2      NULL,
  updatedDate          DATETIME2      NULL,

  CONSTRAINT PK_fact_sales
     PRIMARY KEY NONCLUSTERED (Sale_ID) NOT ENFORCED
)
WITH
(
  DISTRIBUTION = HASH(Product_ID),
  CLUSTERED COLUMNSTORE INDEX
);
GO


COPY INTO dbo.fact_sales
FROM 'https://adlsg2rag.dfs.core.windows.net/gold/export/fact_sales_v2/'
WITH
(
  FILE_TYPE = 'PARQUET',
  CREDENTIAL = (
     IDENTITY = 'Managed Identity'
  )
);
GO
```

Table Features

- Clustered Columnstore Index
- Primary Key:
  NONCLUSTERED NOT ENFORCED

Relationships (Logical)

- fact_sales.Product_ID → dim_product.Product_ID
- fact_sales.SalesRep_ID → dim_salesrep.SalesRep_ID

```
SELECT TOP 100
```

```
  f.Sale_ID,
  f.Sale_Date,
  f.Product_ID,
  p.Product_Category,
  p.Unit_Cost,
  p.Unit_Price,
  f.SalesRep_ID,
  r.Sales_Rep,
  r.Region,
  f.Sales_Amount,
  f.Quantity_Sold,
  f.Customer_Type,
  f.Discount,
  f.Payment_Method,
  f.Sales_Channel
FROM dbo.fact_sales f
LEFT JOIN dbo.dim_product p
  ON f.Product_ID = p.Product_ID
LEFT JOIN dbo.dim_salesrep r
  ON f.SalesRep_ID = r.SalesRep_ID;
GO
```

Neither Primary Keys or Foreign keys are not supported in dedicated SQL pool.

7. Data Loading Strategy

Step 1: Export from Serverless

- Used **CETAS**
- Converted Delta → Parquet

Step 2: Load into Dedicated SQL Pool

- Used COPY INTO

```
COPY INTO dbo.fact_sales
FROM 'ADLS path'
WITH (FILE_TYPE = 'PARQUET');
```

8. Data Validation

Row Count Validation

```
SELECT COUNT(*) FROM dbo.fact_sales;
```

Join Validation

```
SELECT *

FROM fact_sales f

JOIN dim_product p ON f.Product_ID = p.Product_ID

JOIN dim_salesrep r ON f.SalesRep_ID = r.SalesRep_ID;
```

Aggregation

```
SELECT Region, SUM(Sales_Amount)

FROM fact_sales

GROUP BY Region;
```

9. Challenges Faced & Learnings

- Data type mismatches:
  - DOUBLE vs DECIMAL
  - INT vs BIGINT
- Synapse UI confusion:
  - Lake DB vs SQL DB
- Schema mismatch during COPY INTO
- Non-unique business key (Product_ID)
- Implementing SCD1 using Data Flow (no MERGE)
- Importance of **schema consistency across layers**
- Understanding **Delta vs Parquet**
- Dedicated SQL Pool limitations:
  - No FK constraints
- MPP distribution strategies:
  - Hash vs Replicate vs Round-robin

10. Conclusion

This project demonstrates an end-to-end data engineering pipeline using Azure Synapse, including:

- Data ingestion
- Transformation with SCD logic
- Delta Lake processing
- Data warehousing in dedicated SQL pool